

Thought for 4s

בטח. הנה תוכנית העבודה המפורטת כאן בשיחה, כדי שתוכל לעבוד לפיה צעד־צעד.

תוכנית עבודה מפורטת — data_project

פרטי הפרויקט

שם הפרויקט: data_project

לוח: Zybo Z7-20

כלי עבודה: Vivado, Verilog, Vitis C, Python ובהמשך

קטנה שמייצרת או קולטת נתונים, אורזת אותם FPGA/SoC **מטרת הפרויקט:** בניית מערכת ואז AXI DMA בעזרת DDR שולחת אותם ל־CDC, FIFO כחבילות, מעבירה אותם דרך. קוראת ומאמתת אותם בתוכנה.

מטרת העל של הפרויקט

המטרה היא לבנות תיק עבודות מקצועי שמראה שאתה יודע לעבוד עם נושאים רלוונטיים לתעשייה:

- Verilog RTL כתיבת
- FSM — מערכות מצבים
- וזיכרונות פנימיים FIFO
- CDC — מעבר בין שעונים
- AXI-Stream
- AXI DMA
- DDR
- Vivado Block Design
- Vitis C
- לכל בלוק Testbench
- מקצועי GitHub תיעוד

עקרונות עבודה

1. אתה כותב את הקוד

בעצמך testbench את ה־ Verilog המטרה היא שאתה תכתוב את קבצי ה־

אני אעזור ב:

- הסבר מה צריך לכתוב
- תכנון כניסות ויציאות
- פירוק הבעיה לשלבים
- FSM הכוונה לכתיבת
- בדיקת קוד שכתבת
- עזרה בהבנת שגיאות
- testbench עזרה בתכנון

העיקרון: **אתה כותב, אני מכוון ובודק**.

2. testbench לכל בלוק יש

לא עוברים שלב לפני שיש:

- הבנה של מטרת הבלוק
 - Verilog קובץ
 - testbench קובץ
 - סימולציה שעוברת
 - תיעוד קצר
 - commit ל־GitHub
-

הפרויקט נבנה בהדרגה.3

CDC-DDR לא מתחילים ישר מ-

מתחילים מבלוקים פשוטים, בודקים כל בלוק לבד, ואז מחברים למערכת מלאה.

תמונת המערכת הסופית

Data Generator / Communication Receiver



Packet Builder FSM



FIFO



Async FIFO / CDC



AXI-Stream Master



AXI DMA



DDR Memory



Vitis C Program



UART Terminal / Python / CSV / Graph

?מה המערכת עושה בסוף

המערכת לא ממיינת נתונים

אם נכנס:

1, 7, 8, 3, 6, 3

אז נצפה לקבל:

1, 7, 8, 3, 6, 3

באותו סדר בדיוק.

הבדיקה היא:

- האם כל הנתונים הגיעו?
 - האם הסדר נשמר?
 - האם לא השתבש אף נתון?
 - תקין checksum האם?
 - FIFO overflow? האם לא היה?
 - FIFO underflow? האם לא היה?
 - סיים ללא שגיאה DMA האם?
 - DDR? האם הנתונים נשמרו נכון ב־?
-

פורמט החבילה

בשלב מתקדם הנתונים יארזו כך:

Word 0: Header

Word 1: Length

Word 2: Data[0]

Word 3: Data[1]

...

Word N: Data[N-1]

Word N+1: Checksum

דוגמה:

Header = 0xA5A50001

Length = 6

Data = 1, 7, 8, 3, 6, 3

Checksum = 28

וסביבת עבודה GitHub שלב 0 — הקמת

מטרת השלב

להקים את הפרויקט בצורה מסודרת כדי שהוא ישמש כתיק עבודות מקצועי.

מה עושים בשלב הזה

- data_project יוצרים תיקייה בשם
- repository ב-GitHub פותחים
- יוצרים מבנה תיקיות מסודר
- ראשוני README מוסיפים
- מוסיפים מסמך תוכנית עבודה
- מוסיפים סכמת בלוקים
- ראשון commit עושים

למה השלב הזה חשוב

פרויקט טוב לתיק עבודות הוא לא רק קוד שעובד.
צריך שהוא יהיה מסודר, מתועד וברור למעסיק.

מה הקבצים והתיקיות משמשים

rtl/ קבצי Verilog
tb/ קבצי testbench
vivado/ קבצי Vivado, constraints ותיעוד block design
vitis/ שרץ על המעבד C קוד
python/ גרפים CSV, קוד להצגת נתונים
docs/ תיעוד
images/ תמונות וסכמות
reports/ וסימולציות timing, utilization דוחות

למה דווקא משתמשים במבנה כזה

כי זה מבנה שמראה סדר הנדסי
קוד לחוד, בדיקות לחוד, תיעוד לחוד, ותוצאות לחוד.

תוצר בסוף השלב

עם מבנה תיקיות ותיעוד בסיסי ב-GitHub מסודר ב-repository

Definition of Done

השלב גמור כאשר:

- data_project קיימת תיקיית
- repository ב-GitHub יש

- יש README
 - יש תיקיות מסודרות
 - ראשון commit יש
-

שלב 1 — הגדרת פרוטוקול הנתונים

מטרת השלב

.להגדיר בדיוק מה המידע שעובר במערכת ואיך נבדוק שהוא עבר נכון

מה עושים בשלב הזה

מגדירים:

- רוחב נתון: 32 ביט
- מספר דגימות התחלתי
- Header
- Length
- Data
- Checksum
- מה נחשב פלט תקין

למה השלב הזה חשוב

.לפני שכותבים חומרה חייבים לדעת מה החומרה אמורה להוציא.
אי אפשר לבדוק שום דבר expected output בלי

למה השלב הזה משמש

.זהו שלב ארכיטקטורה.
הוא מגדיר את "השפה" של המערכת.

Header, Length ו-Checksum למה דווקא משתמשים בפרוטוקול עם

:כי זה פשוט אבל מקצועי

- מסמן תחילת חבילה Header

- אומר כמה נתונים יש Length
- הם הנתונים עצמם Data
- מאפשר בדיקת תקינות בסיסית Checksum

מה מתעדים

מתעדים docs/01_architecture.md בקובץ

- פורמט החבילה
- רוחב הנתונים
- דוגמת חבילה
- checksum חישוב
- קריטריון PASS/FAIL

תוצר בסוף השלב

מסמך קצר שמגדיר את פרוטוקול הנתונים.

Definition of Done

השלב גמור כאשר אתה יודע לענות

?מה נכנס למערכת

?מה יוצא מהמערכת

?איך נראית חבילה

?איך בודקים שהיא תקינה

2 — Data Generator שלב

מטרת השלב

ראשון שמייצר סדרת נתונים ידועה מראש Verilog לבנות בלוק

מה הבלוק עושה

הבלוק מוציא נתונים כמו

1, 7, 8, 3, 6, 3

או בהמשך

0, 1, 2, 3, 4, 5...

למה צריך את הבלוק הזה

חיישן או מצלמה, UART, SPI, בהתחלה אין לנו.
לכן אנחנו צריכים מקור נתונים פנימי כדי לבדוק את שאר המערכת.

Data Generator לממה דווקא משתמשים ב־

כי הוא מאפשר לבדוק את המערכת בצורה מבוקרת.
אנחנו יודעים מראש מה אמור לצאת, ולכן קל לזהות שגיאות.

מה הבלוק מלמד

- clock ו־reset עבודה עם
- counter או index
- data_valid
- done
- Verilog סדר פעולות בסיסי ב־

RTL קובץ

rtl/data_generator.v

כניסות ויציאות רעיוניות

כניסות:

clk
reset
start

יציאות:

data_out
data_valid
done
index

Testbench

tb/tb_data_generator.v

testbench בודקים ב־

- כל האותות במצב תקין reset אחרי
- מתחיל את יצירת הנתונים start
- הנתונים יוצאים בסדר הנכון
- עולה רק כשיש נתון תקף data_valid
- עולה בסוף done
- done אין נתונים מיותרים אחרי

מה אמורים לראות בסימולציה

Expected: 1, 7, 8, 3, 6, 3

Actual: 1, 7, 8, 3, 6, 3

Result: PASS

תוצר בסוף השלב

עובד ומאמת בסימולציה Data Generator

3 שלב — Packet Builder FSM

מטרת השלב

לבנות מערכת מצבים שמארזת את הנתונים לחבילה מסודרת.

מה הבלוק עושה

ומוציא Data Generator הבלוק לוקח נתונים מה־

Header

Length

Data[0]

Data[1]

...

Checksum

למה צריך את הבלוק הזה

במערכות אמיתיות לא שולחים סתם רצף מספרים.
שולחים חבילות מסודרות כדי שהצד המקבל ידע להבין ולבדוק את המידע.

FSM למה דווקא משתמשים ב־

כי בניית חבילה היא פעולה של שלבים

Header שלח

Length שלח

Data שלח

Checksum חשב ושלח

סיים

היא הדרך הטבעית לנהל רצף כזה בחומרה FSM.

למה הבלוק הזה חשוב לתעשייה

הוא אחד הנושאים הכי נפוצים בראיונות ובתכנון חומרה FSM.
מבוסס על מערכת מצבים ASIC או FPGA ב־controller כמעט כל

מומלצים FSM מצבי

IDLE

SEND_HEADER

SEND_LENGTH

SEND_DATA

SEND_CHECKSUM

DONE

RTL קובץ

rtl/packet_builder_fsm.v

כניסות ויציאות רעיוניות

כניסות:

clk

reset

start

data_in

data_valid

יציאות:

packet_word
packet_valid
packet_done
state/debug_state

Testbench

tb/tb_packet_builder_fsm.v

testbench בודקים ב-

- מעבר נכון בין מצבים
- יוצא ראשון Header
- יוצא שני Length
- הנתונים יוצאים לפי הסדר
- מחושב נכון Checksum
- עולה בסוף done

פלט צפוי לדוגמה

0xA5A50001
0x00000006
0x00000001
0x00000007
0x00000008
0x00000003
0x00000006
0x00000003
0x0000001C

תוצר בסוף השלב

שמייצר חבילה תקינה ומאומת בסימולציה FSM בלוק.

סינכרוני FIFO — שלב 4

מטרת השלב

פנימי ששומר נתונים זמנית בין שני בלוקים buffer להוסיף.

מה הבלוק עושה

הוא תור FIFO:

First In, First Out

מה שנכנס ראשון, יוצא ראשון.

אם נכנס:

1, 7, 8, 3

יוצא:

1, 7, 8, 3

FIFO למה צריך

הבלוק שמייצר נתונים והבלוק שקולט נתונים לא תמיד עובדים באותו קצב.

לדוגמה:

רוצה לשלוח נתון Packet Builder

לא מוכן כרגע AXI-Stream Master

מחזיק את הנתון עד שהצד הבא מוכן FIFO

FIFO למה דווקא משתמשים ב־

כי זו הדרך המקובלת בתעשייה לחבר בין בלוקים עם קצבי עבודה שונים בלי לאבד מידע.

מה הבלוק מלמד

- זיכרון פנימי
- write pointer
- read pointer
- full
- empty
- overflow
- underflow
- count

RTL קובץ

rtl/sync_fifo.v

כניסות ויציאות רעיוניות

כניסות:

clk
reset
wr_en
rd_en
din

יציאות:

dout
full
empty
count
overflow
underflow

Testbench

tb/tb_sync_fifo.v

testbench בודקים ב-

- כתיבה רגילה
- קריאה רגילה
- שמירת סדר
- full מצב
- empty מצב
- overflow
- underflow

תוצר בסוף השלב

שמוכיח שהמידע נשמר ויוצא באותו סדר FIFO.

5 — AXI-Stream Master

מטרת השלב

AXI DMA כדי לחבר ל־AXI-Stream לבנות בלוק שמוציא את הנתונים בפרוטוקול

מה הבלוק עושה

AXI-Stream: ושולח אותם החוצה בעזרת סיגנלי FIFO הבלוק קורא נתונים מה־

tdata
tvalid
tready
tlast

למה צריך את הבלוק הזה

invalid־ data_out לא מקבל סתם AXI DMA.
תקני AXI-Stream הוא מצפה לממשק

AXI-Stream למה דווקא משתמשים ב־

IP־ DSP, וידאו, DMA במיוחד מול, FPGA כי זה פרוטוקול מאוד נפוץ להעברת זרמי נתונים ב־
של Xilinx/AMD.

מה הבלוק מלמד

- handshake
- backpressure
- tvalid
- tready
- tlast
- יציב כאשר הצד השני לא מוכן data שמירת

כלל חשוב

ניתן עובר רק כאשר

tvalid = 1
tready = 1

אם:

```
tvalid = 1  
tready = 0
```

tdata. אז אסור לשנות את

RTL קובץ

rtl/axis_stream_master.v

כניסות ויציאות רעיוניות

כניסות:

```
clk  
reset  
fifo_dout  
fifo_empty  
m_axis_tready
```

יציאות:

```
fifo_rd_en  
m_axis_tdata  
m_axis_tvalid  
m_axis_tlast  
transfer_done
```

Testbench

tb/tb_axis_stream_master.v

testbench בודקים ב-

- תמיד 1 tready העברה כאשר
- יורד ל-0 tready עצירה כאשר
- tdata נשאר יציב בזמן tready=0
- tlast עולה רק במילה האחרונה
- כל המילים יוצאות לפי הסדר

תוצר בסוף השלב

DMA. שעובד נכון בסימולציה ומוכן לחיבור ל־AXI-Stream Master

בסימולציה, שעון אחד Top RTL — שלב 6

מטרת השלב

Vivado Block Design אחת, עדיין בלי RTL לחבר את כל הבלוקים שכתבנו למערכת.

מה מחברים

Data Generator

→ Packet Builder FSM

→ Sync FIFO

→ AXI-Stream Master

למה צריך את השלב הזה

צריך לוודא שהחומרה שלך עובדת בסימולציה, Vitis ו-DDR, DMA לפני שנכנסים ל-.

Vivado למה דווקא עושים את זה לפני

כדי לבודד תקלות.

DMA, clocks, אם הכל עובד בסימולציה, ובחומרה לא — נדע שהבעיה כנראה באינטגרציה או software.

קבצים

rtl/data_project_top.v

tb/tb_data_project_top.v

testbench בודקים ב-

- מפעיל את כל המערכת start
- החבילה נוצרת
- FIFO הנתונים עוברים דרך
- מוציא את החבילה AXI-Stream
- בסוף tlast
- הסדר נשמר

תוצר בסוף השלב

מלא שעובד בסימולציה בשעון אחד Top RTL.

AXI DMA עם Vivado Block Design — שלב 7

CDC ללא DDR

מטרת השלב

DDR-AXI DMA אמיתית עם Zynq שלך למערכת RTL לחבר את ה-

Vivado מה מוסיפים ב-

- Zynq7 Processing System
- AXI DMA
- AXI Interconnect / SmartConnect
- Processor System Reset
- Custom RTL IP
- PS דרך ה- DDR
- לפי הצורך Clocking

מה כל בלוק משמש

Zynq PS

ושולט במערכת Vitis C המעבד שמריץ את תוכנת

AXI DMA

DDR אל AXI-Stream הבלוק שמעביר את הנתונים מ-

DDR

הזיכרון הגדול שבו נשמרים הנתונים

AXI Interconnect / SmartConnect

במערכת AXI מחבר בין רכיבי

Custom RTL IP

Verilog החומרה שאתה כתבת ב-

AXI DMA למה דווקא משתמשים ב־

בלי שהמעבד DDR אל PL הוא הדרך הנכונה והמקובלת להעביר זרם נתונים מה־DMA כי ה־יעתיק כל מילה ידנית.

CDC למה עדיין בלי

כדי לא להוסיף יותר מדי מורכבות בבת אחת:
קודם מוכיחים

RTL → AXI-Stream → AXI DMA → DDR

תוצר בסוף השלב

שמתחבר, מסונתז, ומוכן להרצה על הכרטיס Vivado ב־Block Design

בסיסי Vitis C — שלב 8

מטרת השלב

DMA ומפעילה את ה־ARM שרצה על המעבד C לכתוב תוכנת

מה התוכנה עושה

- AXI DMA מאתחלת את
- buffer מגדירה כתובת DDR ב־
- לקבל bytes כמה DMA אומרת ל־
- מחכה לסיום ההעברה
- DDR קוראת את הנתונים מה־
- Header, Length, Data, Checksum בודקת
- PASS/FAIL מדפיסה

למה צריך את התוכנה

החומרה לא מתחילה לבד.

DDR ולבדוק מה נכנס ל־DMA צריך להפעיל את ה־PS ה־

Vitis C למה דווקא

Zynq כי זה כלי הפיתוח הסטנדרטי לתוכנה שרצה על המעבד של

מה התוכנה מוכיחה

בכרטיס DDR שהמערכת לא רק עובדת בסימולציה, אלא באמת כותבת ל-

תוצר בסוף השלב

בטרמינל נראה משהו כמו:

```
DMA transfer completed
Header OK
Length OK
Data OK
Checksum OK
Verification PASSED
```

9 CDC ו-Async FIFO — שלב

מטרת השלב

להוסיף מעבר בטוח בין שני שעונים שונים.

מה הבלוק עושה

מאפשר Async FIFO:

כתיבה בשעון אחד
קריאה בשעון אחר

לדוגמה:

Data Clock = 50MHz
AXI Clock = 100MHz

למה צריך את הבלוק הזה

clock domains במערכות אמיתיות יש הרבה.

של נתונים בין שעונים שונים בצורה ישירה bus אי אפשר להעביר.

Async FIFO למה דווקא משתמשים ב־

הוא פתרון סטנדרטי ובטוח Async FIFO, בין שעונים שונים data bus כי עבור מעבר של

מה הבלוק מלמד

- CDC
- wr_clk
- rd_clk
- clock domains בשני empty/full
- למה צריך סנכרון
- שמירת סדר הנתונים בין שעונים

RTL קובץ

rtl/async_fifo.v

Testbench

tb/tb_async_fifo.v

testbench בודקים ב־

- שונים clocks שני
- כתיבה בצד אחד
- קריאה בצד השני
- הסדר נשמר
- אין איבוד נתונים
- עובדים full/empty

תוצר בסוף השלב

שעובד בסימולציה עם שני שעונים שונים Async FIFO.

CDC + DMA + DDR שלב 10 — אינטגרציה מלאה עם

מטרת השלב

clock domains למערכת המלאה ולעבוד עם שני Async FIFO להכניס את

מה המערכת עושה עכשיו

Data Generator

→ Packet Builder FSM

→ Async FIFO / CDC

→ AXI-Stream Master

→ AXI DMA

→ DDR

→ Vitis C

למה השלב הזה חשוב

:זה השלב שבו הפרויקט נהיה ממש דומה למערכת תעשייתית

- שלך RTL יש
- CDC יש
- DMA יש
- DDR יש
- יש תוכנה
- יש בדיקת מערכת מלאה

בשלב הזה CDC למה דווקא משלבים

.כי קודם וידאנו שכל חלק עובד לבד

.עכשיו אפשר להוסיף מורכבות בצורה מבוקרת

מה בודקים

- עובדים clocks שני
- overflow אין
- underflow אין
- באותו סדר DDR הנתונים מגיעים ל-

- checksum תקין
- מסיים בהצלחה DMA

תוצר בסוף השלב

שעובדת על הכרטיס DDR ו־CDC מערכת מלאה עם

Vivado ודוחות ILA — שלב 11

מטרת השלב

אמיתית על הכרטיס ולתעד תוצאות הנדסיות debug להוסיף יכולת

עושה ILA מה

בזמן אמת FPGA פנימי שמאפשר לראות סיגנלים בתוך ה־ logic analyzer הוא ILA

ILA למה צריך

כי בחומרה אמיתית לא תמיד רואים מה קורה

מאפשר לבדוק סיגנלים כמו ILA

tdata
tvalid
tready
tlast
fifo_empty
fifo_full
state
done

ILA למה דווקא משתמשים ב־

FPGA. כי זה כלי דיבוג מקצועי ונפוץ מאוד בעבודה עם

מה עוד מתעדים

- utilization report
- timing summary
- WNS

- LUT usage
- FF usage
- BRAM usage

תוצר בסוף השלב

יהיו reports בתיקיית

- ILA צילום
 - timing report
 - utilization report
 - הסבר קצר על התוצאות
-

שלב 12 — Python / CSV / Graph

מטרת השלב

.להציג את הנתונים בצורה ברורה ויפה לתיק עבודות

עושה Python מה

- קורא את הפלט מהמערכת
- CSV שומר
- מצייר גרף
- מאפשר להציג תוצאה ויזואלית

למה צריך את השלב הזה

:כי למעסיק יותר קל להבין פרויקט כשיש תוצאה ויזואלית

FPGA → DDR → C → Python → Graph

Python למה דווקא

.כי הוא נפוץ, פשוט, ומתאים מאוד לניתוח נתונים והצגה

תוצר בסוף השלב

- CSV קובץ
 - גרף
 - צילום מסך
 - GitHub תיעוד ב־
-

שלב 13 — הרחבה לרכיב תקשורת אמיתי

מטרת השלב

הפנימי במקור נתונים אמיתי Data Generator להחליף את.

UART Receiver — A אפשרות

UART למה להשתמש ב־

- קל לבדוק מהמחשב
- מתאים לטרמינל
- פשוט יחסית
- טוב ללימוד פרוטוקול סדרתי

למה הוא משמש

FPGA לקבלת נתונים מהמחשב אל ה־

SPI Master / SPI Receiver — B אפשרות

SPI למה להשתמש ב־

- נפוץ מאוד מול חיישנים
- embedded מתאים לעולם
- מעניין וברור FSM דורש
- רלוונטי לתעשייה

למה הוא משמש

או חיישן ADC לקבלת או שליחת נתונים לרכיב חיצוני, כמו.

למה לא מתחילים מזה

כי בהתחלה עדיף לבודד את הבעיה.
אחרי שהמערכת הפנימית עובדת, קל יותר להוסיף תקשורת.

תוצר בסוף השלב

מערכת שבה הנתונים מגיעים ממקור תקשורת אמיתי ולא רק ממחולל פנימי

שלב 14 — ליטוש תיק עבודות

מטרת השלב

להפוך את הפרויקט למשהו שניתן לשלוח למעסיקים.

מה עושים

- README משפרים
- מוסיפים סכמת בלוקים
- waveform מוסיפים תמונות
- ILA מוסיפים צילום
- Vivado מוסיפים דוחות
- מסבירים כל בלוק
- כותבים מה למדת
- כותבים הרחבות עתידיות

למה השלב הזה חשוב

מעסיק לא יקרא את כל הקוד מהתחלה.
הוא צריך להבין מהר:

מה בנית
למה זה חשוב
מה אתה יודע
מה הייתה התרומה שלך

תוצר בסוף השלב

מסודר שמציג את הפרויקט כתיק עבודות מקצועי GitHub.

סדר העבודה הסופי

0. GitHub + project structure
 1. Data protocol
 2. Data Generator + Testbench
 3. Packet Builder FSM + Testbench
 4. Sync FIFO + Testbench
 5. AXI-Stream Master + Testbench
 6. Full RTL Top Simulation
 7. Vivado Block Design + AXI DMA + DDR
 8. Vitis C Verification
 9. Async FIFO + CDC + Testbench
 10. Full Integration with CDC + DMA + DDR
 11. ILA + Vivado Reports
 12. Python CSV / Graph
 13. UART or SPI Extension
 14. Portfolio Polish
-

לפני מעבר שלב Check List

לפני שעוברים שלב, חייבים לענות כן על כל השאלות

- האם אני מבין מה מטרת השלב?
- האם אני מבין מה הבלוק עושה?
- האם אני מבין למה משתמשים בבלוק הזה?
- בעצמי Verilog האם כתבתי?
- testbench האם כתבתי?
- האם הסימולציה עברה?
- האם תיעדתי את התוצאה?
- commit ל-GitHub האם עשיתי?
- האם אני יודע להסביר את השלב בראיון?

אם אחת התשובות היא לא — לא עוברים שלב.

משפט אפשרי לקורות חיים

Designed and implemented a Zynq-based DDR data logger on Zybo Z7-20 using custom Verilog RTL, FSM-based packet generation, FIFO buffering, clock-domain crossing, AXI-Stream, AXI DMA, DDR memory, Vitis C validation, and Python-based visualization.

סיכום

FPGA/SoC הוא פרויקט שנבנה בהדרגה כדי להראות יכולת אמיתית בעולם data_project.

הפרויקט כולל:

שאתה כותב בעצמך RTL

לכל בלוק testbench

Vivado Block Design

עם AXI DMA עבודה

שימוש ב-DDR

CDC

Vitis C

להצגה Python

מקצועי GitHub תיעוד

המטרה היא לא רק שהפרויקט יעבוד, אלא שתוכל להסביר אותו היטב בראיון ולהציג אותו כתיק עבודות.